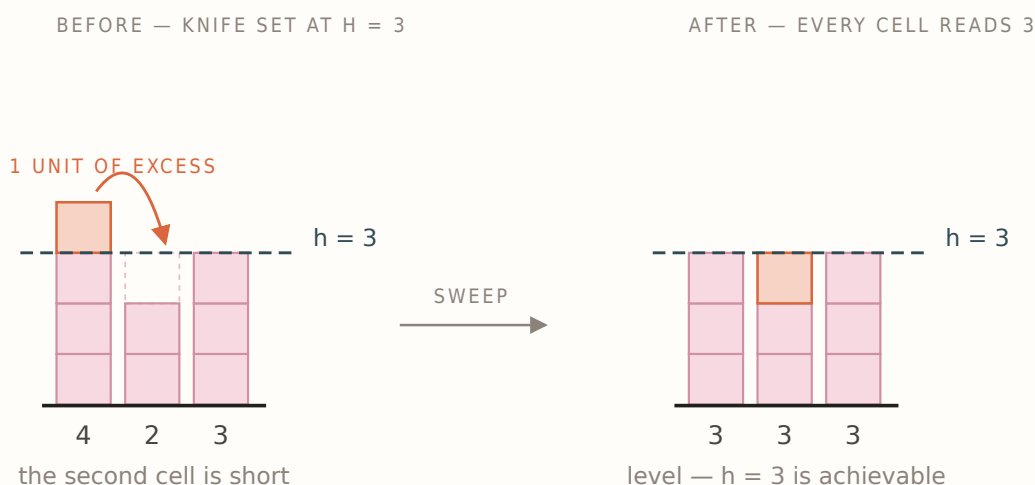


Cake Frosting

THE TALLEST LEVEL HEIGHT OF EVERY PREFIX · HOW THE SOLUTION WORKS

Alice sets her knife at an integer height h and sweeps once from left to right. Whatever a cell holds above h is pushed onto the next cell; what spills past the last cell is lost. For each prefix length i we want the largest h that leaves cells $1 \dots i$ all sitting at exactly h .

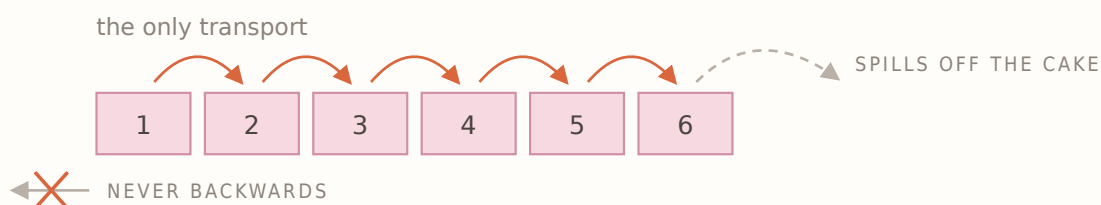
THE OPERATION



The array $[4, 2, 3]$. At $h = 4$ the middle cell has no way to reach the knife and the cake stays uneven. At $h = 3$ the first cell sheds one unit, that unit lands exactly where it was needed, and all three cells finish level.

The sweep only ever takes frosting away from a cell that is too tall. It cannot invent frosting for a cell that is too short. So “level at h ” is a genuine constraint, and lowering the knife is the only lever we have.

EVERYTHING DRIFTS RIGHT, NOTHING DRIFTS LEFT



Because frosting moves in one direction only, the first j cells are a closed system: nothing to their right can ever reach them.

This is the whole problem in one sentence. Whatever the first j cells are going to hold, they are already holding it. Their total is fixed at $S_j = a_1 + \dots + a_j$, no matter which h Alice picks, and no matter how the sweep plays out.

WHAT A PREFIX CAN AFFORD

Finishing the first j cells at height h costs $h \cdot j$ units of frosting, and that bill has to be paid out of S_j alone. If the prefix is short, no choice of sweep order rescues it — the frosting simply is not in the room.



The prefix $[4, 2]$, holding six units. The dashed box is the frosting a level finish would require. Pouring the six units into the box and letting them find their own level is precisely what the sweep does.

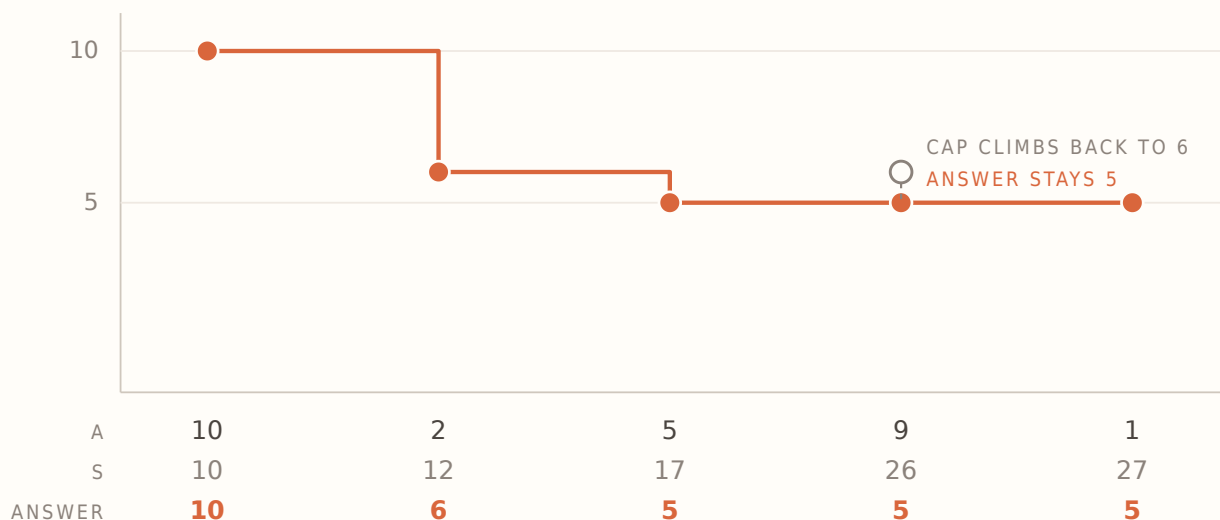
AND IT IS ENOUGH. The condition is not merely necessary. Sweep with an h that satisfies it everywhere, and the surplus handed from prefix j to the next cell is exactly $S_j - h \cdot j$, which the condition keeps at or above zero. A cell is starved only when that quantity would go negative, so no cell is ever starved, and the prefix comes out flat.

$$S_j \geq h \cdot j \text{ for every } j \leq i \Leftrightarrow h \leq \lfloor S_j/j \rfloor \text{ for every } j \leq i$$

Read the right-hand form as a ceiling that each prefix imposes on the knife. Call $\lfloor S_j/j \rfloor$ the *cap* of prefix j — the average height of the first j cells, rounded down because the knife sits at an integer.

THE ANSWER IS A RUNNING MINIMUM

A prefix of length i must respect the cap of every prefix inside it, so the best knife height is the smallest cap seen so far. Growing the array adds constraints and never removes one, which is why the answers never rise as i increases.



Caps for $[10, 2, 5, 9, 1]$ are 10, 6, 5, 6, 5. At $j = 4$ a tall cell lifts that prefix's own cap back to 6, but prefix 3 already pinned the knife at 5 and no later cell can undo it.

WHERE THE CARRY WENT

The instinctive implementation walks the array carrying two pieces of state: the current knife height and a running carry of pushed frosting. When a cell comes up short, the knife drops far enough that the inflow from the left covers it. That works, and it is also the version that is easy to get wrong — the carry leaving a cell is everything that *arrives* minus the amount the cell *keeps*, and dropping either term produces an answer that still passes the samples.

It is also unnecessary. Write the carry out in closed form and the state dissolves:

CARRY ARRIVING AT CELL i

$$S_{i-1} - h \cdot i$$

SO THE FROSTING SITTING ON CELL i IS

$$a_i + \text{carry} = S_i - h \cdot i$$

THE LOCAL TEST FOR A CANDIDATE HEIGHT t

$$S_i - \cancel{h \cdot i} + \cancel{h \cdot i} - t \cdot i \geq t$$

THESE CANCEL



$$S_i \geq t \cdot (i + 1)$$

The knife height h appears twice with opposite signs and leaves the expression entirely. What survives is the prefix cap again — no carry, no binary search, nothing to keep in sync.

THE RECIPE

1. Walk the array once, left to right, holding the prefix sum S .
2. At position i , the cap of this prefix is $\lfloor S/i \rfloor$.
3. Keep the smallest cap met so far; that value is the answer for prefix i . Print it and move on.

SMALL PRINT

One pass, $O(n)$ time and constant extra memory beyond the input. Prefix sums reach $2 \cdot 10^{14}$, comfortably past 32 bits, so the running total wants a 64-bit type; the printed answers themselves never exceed $\max a_i$.

Two corners fall out for free. The answer at $i = 1$ is a_1 , since a single cell is already level. And because every a_i is at least 1, every cap is at least 1, so the running minimum never collapses to zero and the knife always has somewhere legal to sit.